DIGITAL SEARCH TREES

A digital search tree is a binary tree in which each node contains one element. The element to node assignment is determined by the binary representation of the element keys. The digital search tree procedures to search, insert, and delete are quite similar to the corresponding procedures for binary search trees. The essential difference is that the sub-tree to move to is determined by a bit in the search key rather than by the result of the comparison of the search key and the key in the current node.

Each of the above search tree operations can be performed in O(h) time where h is the height of the digital search tree. If each key in a digital search tree has *Key Size* bits, then the height of the digital search tree is at most *Key Size* + 1.

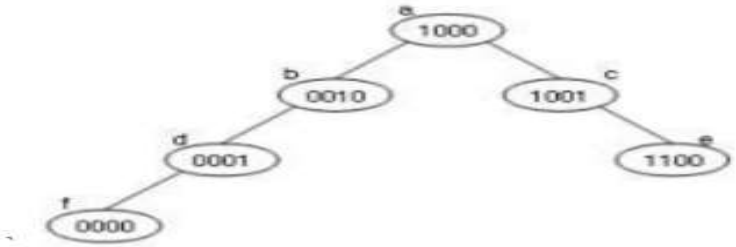


Figure 1: Example digital search tree

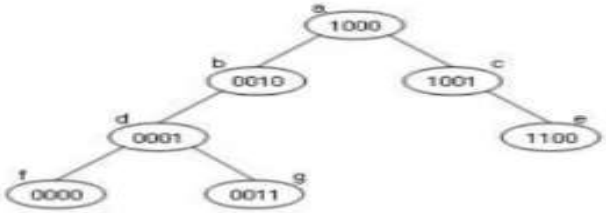


Figure 2: Digital search tree of Figure 1 following insertion of 0011

DEFINITIONS:

A digital search tree (DST) is a binary tree whose ordering of nodes is based on the values of bits in the binary representation of a node's key.

Creating and Searching DSTs:

Let us build a tree with the following elements: 1001, 0110, 0000, 1111, 0100, 0101, 1110. Since we start with an empty tree, the first element, 1001, becomes the root of the tree. The next key to be inserted is 0110. First, we have to check whether this key is equal to the key of the root node. Since it is not, we continue traversal down the tree. To decide where to place the node, we look at the first bit from the left. Since this bit is 0, we take the path down the left subtree and find that the left pointer is NIL. Therefore, we create a new node, which is the left child of the root node, as shown in Figure 1a. The next element to be inserted is 0000. Since the root of the tree is not NIL (and its key is not equal to the key being inserted), we look at the first bit (from the left) of the key to be inserted. Since this bit is 0, we take the path down the left subtree. Since the next node is not NIL either, we look at the next (second) bit of the key to be inserted. The bit is 0, so we take the path down the left subtree. At this point we encounter a NIL pointer and place the new node in its place, as depicted in Figure 1b. We continue the procedure until all nodes have been added, and the tree in Figure 1f is obtained. Bits used to make the decisions while progressing down the tree are shown in bold in the figure. It is worth noting that scanning the bits from left to right, when making search decisions, is an arbitrary choice. The tree can be built in the same manner by scanning the bits from right to left.

Insertion:

The insertion method for DSTs is nearly identical to the insertion method for binary search trees. We first compare the key to be inserted with the key in the current node, except that for DSTs we test only for equality. In addition, DST algorithms test the corresponding bit of the new key to choose the next move. To search for an element, we traverse the tree in the same way the insert procedure does. If along the way we find that the current node's key matches the key we are looking for, the search completes successfully. If during the traversal we reach a NIL node, this means that the search key does not exist in the tree. The above discussion deals with the case where all keys are of equal length, which happens in many applications (e.g. when the keys are integers). The algorithm requires only a minor modification to handle variable length keys. Consider, for example, how the key 11 can be added to the tree in Figure 1f. As before, we follow the right link twice, because the first two bits are 1. At this point we have run out of bits to analyze. In such a case, it is obvious that the node we are currently at has to have a key with a prefix equal to the new key, and has to be longer than the new key (otherwise the two keys are the same). Therefore, we can place the new key (11) in place of the existing node (1110), and continue by inserting the existing node (1110) further down the tree. Since the third bit of 1110 is 1, we go to the right, and place the new node there because we have reached a NIL link. The resulting tree is shown in

Deleting Nodes from DSTs :

Procedure to delete a node in the DST simply replaces the node with any leaf node in either of the node's subtrees (E. Coffman, 1970). If the node to be deleted does not have any children, it is simply removed and replaced by a NIL pointer. For instance, removing the node 0110 from the tree in Figure 1g can be done by replacing it with either node 0000 or 0101. It

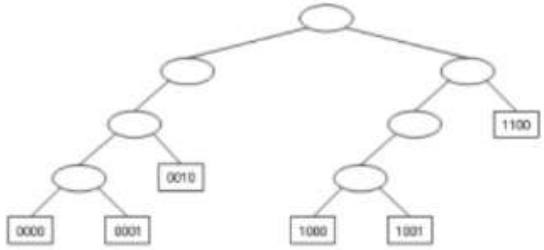
is obviousthat this method is similar, and in fact simpler than the deletion method for ordinary binary trees, which has to consider three distinct cases

BINARY TRIES:

When we have very long keys, the cost of a key comparison is high. We can reduce the number of key comparisons to one by using a related structure called *Patrica*(*P*ractical *a*lgorithm *t*o *r*etrieve *i*nformation *c*oded *i*n *a*lphanumeric). We chall develop this structure in three steps. First, we introduce a structure called a binary trie. Then we transform binary tries into compressed binary tries. Finally, from compressed binary tries we obtain Patricia.

A *binary trie* is a binary tree that has two kinds of nodes *branch nodes* and *element nodes*. A branch node has the two fields *left_child* and *right_child*. It has no *data* field. An element node has the single field *data*. We use branch nodes to build a binary tree search structure similar to that of a digital search tree. This search structure leads to element nodes.

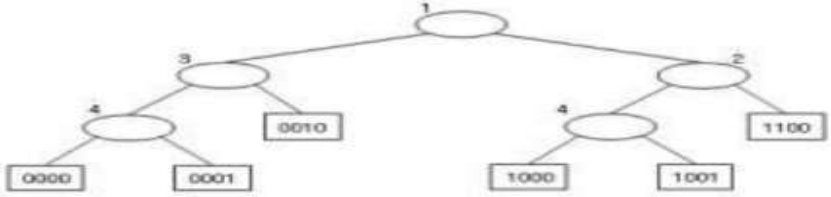
Fig shows a six-element binary trie. To search for an element with key k, we use a branching pattern determined by the bits of k. The ith bit of k is used at level i. If it is zero, the search moves to the left subtree. Otherwise, it moves to the right subtree.



Example binary trie.

Observe that a successful search in a binary trie always ends at an element node. Once this element node is reached, the key in this node is compared with the key we are searching for. This is the only key comparison that takes place. An unsuccessful search may terminate at either an element node or at a *NULL* pointer.

The binary trie of Fig contains branch nodes whose degree is one. By adding another field bit_number field of a branch node gives the bit number of the key that is to be used at this node. Fig gives the binary trie that results from the elemination of degree one branch nodes from the binary trie of Fig. The number outside a node is its bit_number. A binary trie that has been modified in this way to contain no branch nodes of degree one is called a *compressed binary trie*.



Patricia:

Compressed binary tries may be represented using nodes of a single type. The new nodes, called *augmented branch nodes* are the original branch nodes augmented by the field *data*. The resulting structure is called *Patricia* and is obtained from a compressed binary trie in the following way:

(1) Replace each branch node by an augmented branch node.

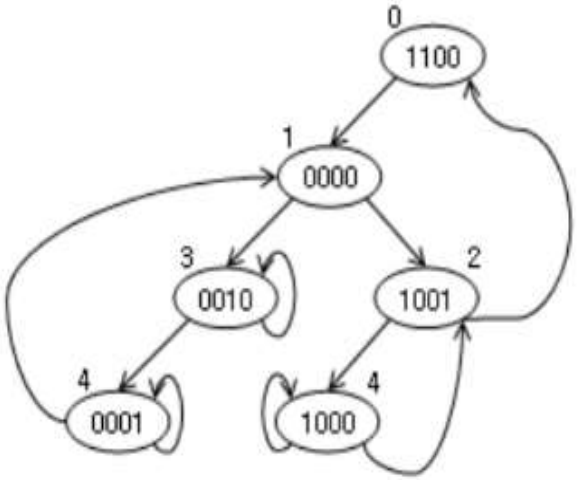
(2) Eliminate the element nodes.

(3) Store the data previously in the element nodes in the data fields of the augmented branch nodes. Since every non empty compressed binary trie has one fewer branch node than it has element nodes, it is necessary to add one augmented branch node. This node is called the *head node*. The remaining structure is the left subtree of the head node has bit_number equal to zero. Its right child field is not used. The assignment of data to augmented branch nodes is done in such a way that the but_number in the augmented branch node is less than or equal to that in the parent of the element node that contained this data.

Replace the original pointers to element nodes by pointers to the respective augmented branch nodes.

Activate Windows

When the above transformations are performed on the compressed trie of Figure, we get the structure of Figure. Let *t* be an instance of Patricia. *t* is *NULL* if the instance is empty. An instance, *t*, with one element is represented by a head node whose left child field points to itself.



An example of Patricia

Activate Windows

Multiway tries:

- A binary trie uses radix search with radix 2; a multiway trie uses radix search with radix $R > 2$
  - multiway tries are sometimes called R-ary tries
- If each digit in a key has r bits, the radix is $R = 2^r$, and if keys have at most B bits, the worst-case number of comparisons would be only B/r
- However, to implement this idea, a node in the trie must be able to have as many as R children
- Examples:
  - Keys are words made up of lower-case letters in English. There are 26 different lower-case letters in English, so a R-ary trie with R=26 could hold these keys. (This specific variant is sometimes called an "alphabet trie")
  - Keys are decimal integers made up of decimal digits. There are 10 different decimal digits, so a R-ary trie with R=10 could hold these keys
  - Keys are 128-bit IEEE high precision floating point numbers. Consider each as made up of 32 4-bit nybbles. There are $2^4 = 16$ different nybble values, so a R-ary trie with R=16 could hold these keys (note that lexicographic ordering of such keys is not the same as their numeric ordering)

Activate Windows

Multiway trie properties

- The structure of a multiway trie depends only on the keys in it, not on the order in which they were inserted
- Multiway tries have a strong key ordering property: At a node X, all keys in X's leftmost subtree are smaller than keys in X's next-to-leftmost subtree, etc. (according to lexicographic ordering)
    - So, a preorder traversal of a multiway trie visits keys in sorted order
    - Also, after following a sequence of digits to get to a node X, all keys in the trie that have that sequence as prefix are in the subtree rooted at X
- Suppose there are r bits per digit (so radix $R = 2^r$), and keys contain at most B bits. Then:
    - The worst case height of a R-ary trie containing N keys is B/r
    - Compare: worst case height in a regular BST containing N keys is N, which with B-bit keys can be as much as 2^B
- When N is large and B is comparable to $\log_R N$, DST's give worst-case time cost guarantees better than balanced BST's, and are much easier to implement
- However there is a space cost disadvantage of multiway trees: Each node must store R child pointers, and for a typical tree many of these will be null
- This problem can be addressed with ternary tries

Insert into a Trie:

The first operation is to insert a new node into the trie. Before we start the implementation, it is important to understand some points:

1. Every letter of the input key (word) is inserted as an individual in the Trie_node. Note that children point to the next level of Trie nodes.
2. The key character array acts as an index of children.
3. If the present node already has a reference to the present letter, set the present node to that referenced node. Otherwise, create a new node, set the letter to be equal to the present letter, and even start the present node with this new node.
4. The character length determines the depth of the trie.

Deletion from a Trie:

The Third operation is the deletion of a node in the Trie. Before we begin the implementation, it is important to understand some points:

1. If the key is not found in the trie, the delete operation will stop and exit it.
2. If the key is found in the trie, delete it from the trie.

- The term trie comes from re**trie**val.
- This term was coined by Edward Fredkin, who pronounce it tri as in the word retrieval.

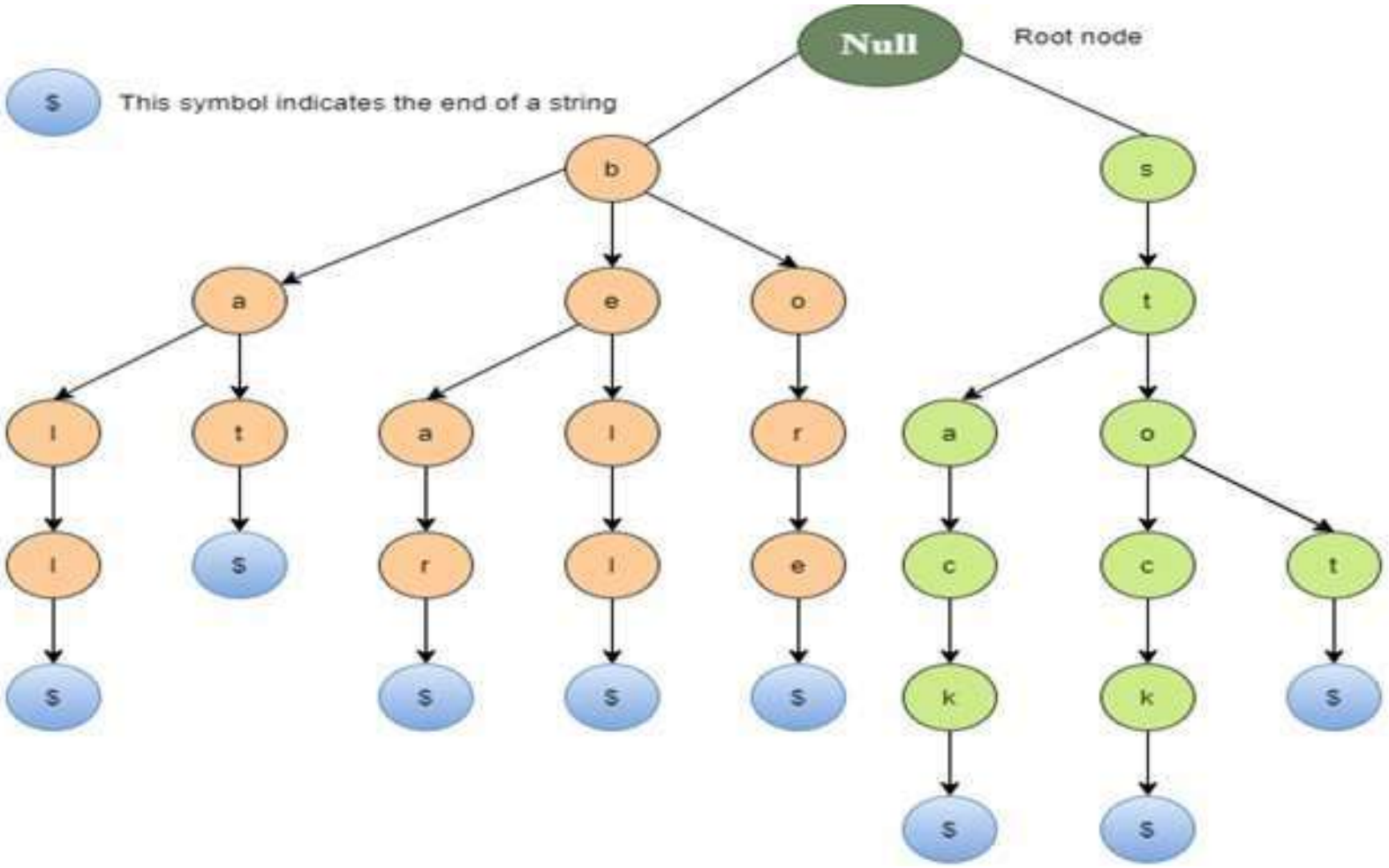
- A data structure for representing a collection of strings.
- In computer science, a trie, also called digital tree and sometimes radix tree or prefix tree.

Tries support fast pattern matching.

Properties of the Trie for a set of the string:

1. The root node of the trie always represents the null node.
2. Each child of nodes is sorted alphabetically.
3. Each node can have a maximum of **26** children (A to Z).
4. Each node (except the root) can store one letter of the alphabet.

Trie representation for the **bell, bear, bore, bat, ball, stop, stock, and stack.**

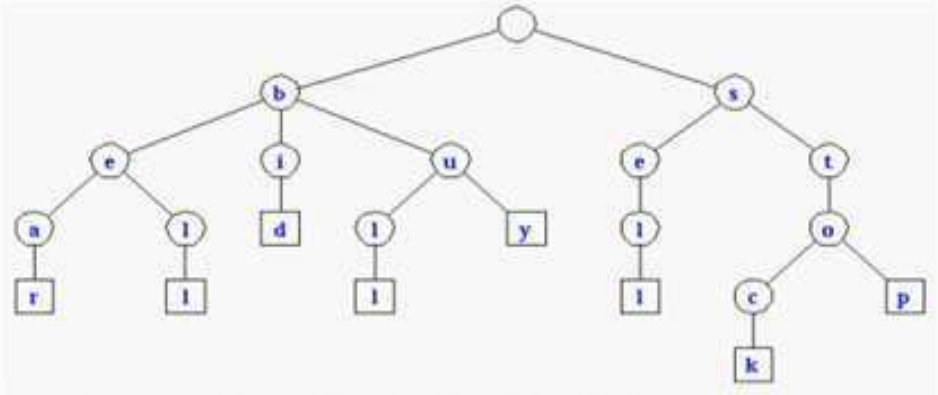


Types of Tries

1. Standard Tries
2. Compressed Tries
3. Suffix Tries

Standard Tries

- The ***standard trie*** for a set of strings S is an ordered tree such that:
  - each node but the root is labeled with a character
  - the children of a node are alphabetically ordered
  - the paths from the external nodes to the root yield the strings of S
- Example: standard trie for the set of strings S = { bear, bell, bid, bull, buy, sell, stock, stop }



•A standard trie uses O(n) space. Operations (find, insert, remove) take time O(dm) each, where:

-n = total size of the strings in S,

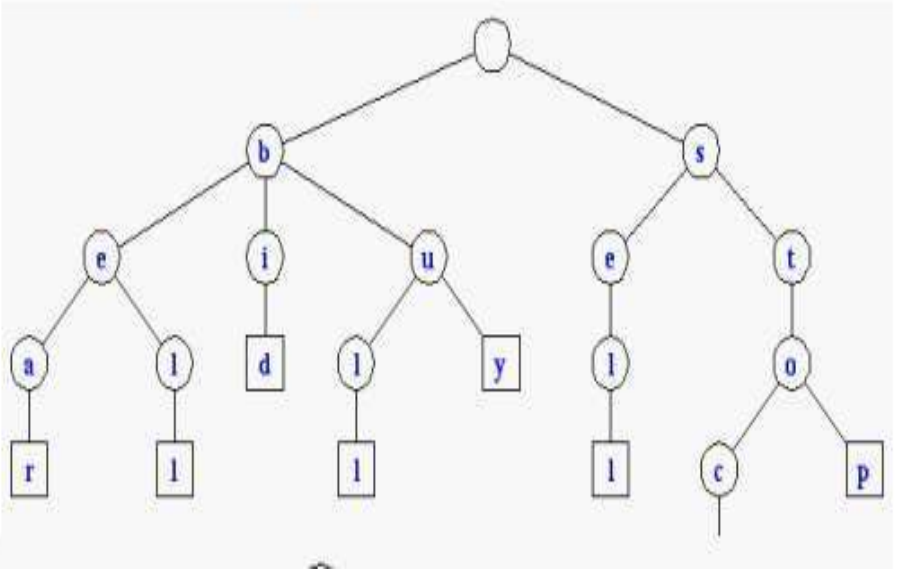
-m =size of the string parameter of the operation

-d =alphabet size

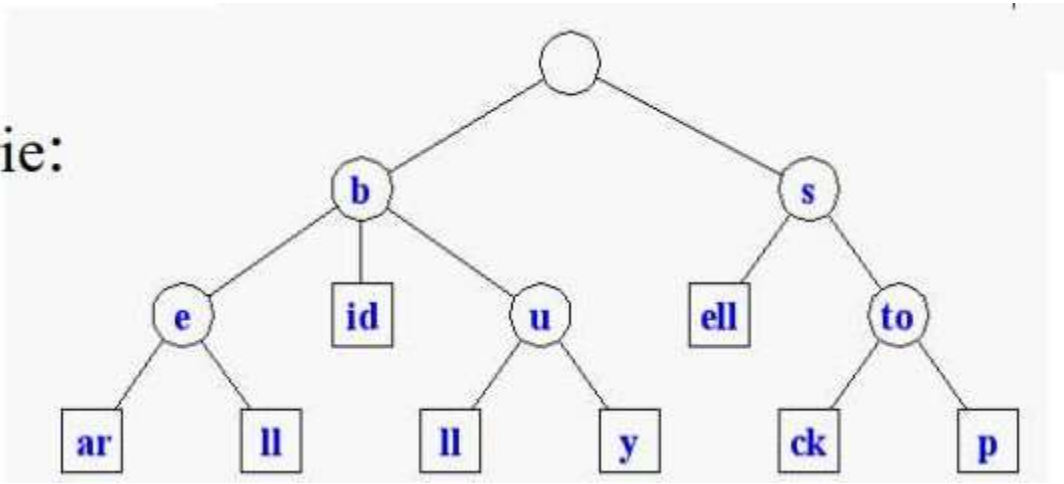
Compressed trie:

- Trie with nodes of degree at least 2
- Obtained from standard trie by compressing chains of *redundant nodes*

Standard Trie:



Compressed Trie:



Suffix trie:

- A suffix trie is a compressed trie for all the suffixes of a text.
- Suffix trie are a space-efficient data structure to store a string that allows many kinds of queries to be answered quickly.

Properties of Suffix Tries

The ***suffix trie*** for a text X of size ***n*** from an alphabet of size ***d***

-stores all the ***n(n-1)/2 suffixes*** of X in O(n) space

-supports arbitrary ***pattern matching*** and prefix matching queries in ***O(dm) time***, where m is the length of the pattern

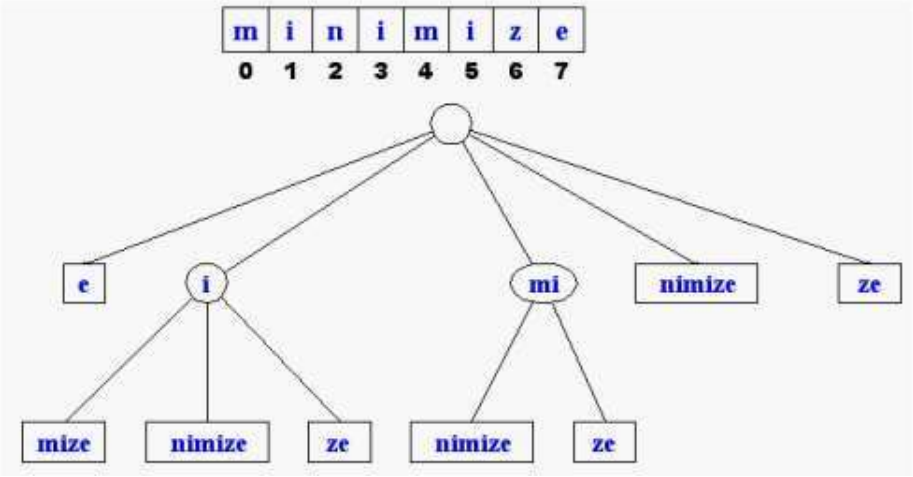
-can be constructed in ***O(dn) time***

m	i	n	i	m	i	z	e
0	1	2	3	4	5	6	7

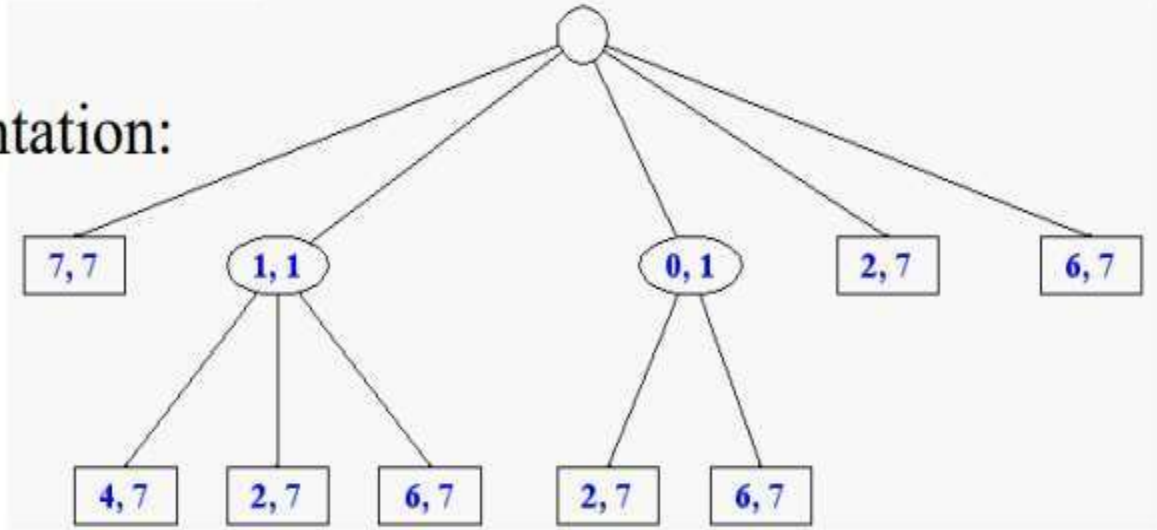
Suffix Tries

- A ***suffix trie*** is a compressed trie for all the suffixes of a text

Example:



Compact representation:



- What is the difference between a tree and a trie?
- A tree is a general structure of recursive nodes. There are many types of trees. Popular ones are binary tree and balanced tree. A Trie is a kind of tree, known by many names including prefix tree, digital search tree, and retrieval tree (hence the name 'trie').

Applications of Trie

1. Spell Checker

2. Auto-complete

3. Browser history

Advantages of Trie

It can be insert faster and search the string than hash tables and binary search trees.

It provides an alphabetical filter of entries by the key of the node.

Disadvantages of Trie

It requires more memory to store the strings.

It is slower than the hash table.